Erasable Substructural Propositions for SStruct

A design for LProp: linear propositions tracked on the ordinary ledger and erased by their universe

Qiancheng Fu

June 19, 2026

Abstract

SStruct is a dependent type theory that unifies the standard substructural disciplines (linear, affine, relevant, unrestricted) through a lattice of sorts, and erases computationally irrelevant arguments in the style of the implicit calculus of constructions. This note works out, by hand, an extension with *erasable substructural propositions* (LProp): a universe of proofs that are tracked under a substructural discipline for reasoning, yet carry no runtime footprint and are removed at extraction. The design keeps a *single* resource ledger. A proof is an ordinary explicit variable whose type is a proposition, so the existing context-splitting machinery tracks it unchanged, and erasure is a separate pass that deletes terms of propositional type, as in the extraction mechanism of Rocq. That form of erasure is sound in Rocq because every value there is discardable. The substructural setting needs one further condition, empty in Rocq but required here: a runtime resource may be consumed into a proof only as a drop, hence only when its sort permits weakening. We give the propositional universes and their placement, the formation, introduction, and elimination rules, the two elimination walls that preserve progress after erasure and confine resources, the action of extraction, and the resulting metatheory. The existing treatment of equality and falsity in SStruct reappears as the singleton and empty instances of the new universe.

Contents

1 Introduction

SStruct already does two things that, taken together, point directly at the feature we want. First, it tracks resources: every type is classified by a sort drawn from a lattice, and the typing context is split, weakened, and contracted according to that sort, so that linear values are used exactly once, affine values at most once, relevant values at least once, and unrestricted values arbitrarily. Second, it erases: arguments marked implicit are computationally irrelevant, are typed in a resource-free *logical* layer, and are replaced by a null placeholder at extraction, as in the implicit calculus of constructions [1].

What SStruct does *not* yet have is a user-facing universe of erasable propositions analogous to `Prop` in Rocq or Lean: a place to declare one's own propositions, to quantify over them impredicatively, and to have their proofs vanish at extraction. The goal of this note is to add such a universe. Because SStruct is substructural, we make the universe substructural too, so that one can state and track *linear* propositions (capabilities, separation-logic assertions, one-shot protocol witnesses) whose proofs are nonetheless erased.

Two erasure mechanisms, kept distinct. A first instinct is to reuse the implicit/explicit marker for proofs: erased things are implicit, so make proofs implicit. This conflates two independent notions. The implicit marker is a per-binder *relevance* decision, and an implicit argument is untracked and used freely in the resource-free layer, so it cannot carry a linear discipline. Erasure of a whole *proposition* need not be a per-binder bit at all. It can be a pass over the typing derivation that deletes every term of propositional type, the way the extraction mechanism of Rocq removes `Prop` [2]. Once the two are separated, a proof can stay an ordinary *explicit*, fully tracked variable during type checking and still be erased, since erasure consults its type rather than its relevance marker. We follow this route. It keeps one ledger and changes nothing about context splitting; the only new run-time machinery is the deletion of propositional terms, and the only new typing discipline is a single side condition.

The substructural delta over Rocq. Deleting propositional terms is sound in Rocq for a reason that does not survive the move to a substructural calculus. Every value in Rocq is unrestricted, so a proof may freely contain other values, and deleting them is harmless because anything discardable may be discarded. In SStruct a value may be linear, and deleting a proof that has captured a linear resource leaks it. The substructural content of the design is a single principle:

A runtime resource is consumed into a proof only as a drop, so only a resource whose sort permits weakening may be so consumed, except inside a provably dead branch.

The principle holds vacuously in Rocq, where every sort permits weakening. It reuses SStruct's existing drop discipline, including the exception that a dead branch may drop anything. It has two enforcement points, a condition on quantifier formation and a condition on elimination into a propositional motive, and these are the only new typing rules.

What is already latent in SStruct. The system contains a fully worked, if special-purpose, erasable-proposition fragment. Propositional equality and the empty type live in the least sort. Their proofs are typed in the logical layer and erased: the rewrite eliminator extracts to its body, and ex-falso extracts to a dead marker, dropping the resources of its unreachable branch. These are the *singleton* and *empty* cases of a general erasable-proposition universe, and they already discharge the two standard concerns with Rocq and Lean. Large elimination out of the empty type is sound because it is uninhabited in any closed configuration, and equality erases because its proofs carry no information. LProp generalizes this fragment to a user-extensible, impredicative, substructural universe, and the existing rules survive as its degenerate instances.

Contributions.

- A propositional universe for each sort (§4): the existing lattice indexes a family $\mathbb{P}_s$ of erasable propositions, placed at the bottom of the level hierarchy and quantified impredicatively. This fixes where an impredicative LProp sits relative to the existing sorts, and gives linear, affine, relevant, and unrestricted propositions uniformly.
- A single-ledger design (§3) in which proofs are ordinary explicit variables of propositional type, tracked by the existing context split, and erased by a universe-directed pass.
- The resource-confinement principle and its two enforcement points (§3, §6), with the leak that motivates each.
- The two elimination walls (§6): one preserving progress after erasure (the singleton-elimination criterion of Rocq and Lean, transported), and one confining resources.
- The action of extraction (§7) and the resulting metatheory (§8), whose two principal obligations are a resource-confinement lemma and consistency of the impredicative universe.

Related systems. The relevance-based erasure descends from the implicit calculus of constructions [1]; the universe-directed erasure of propositions is the extraction mechanism of Rocq [2], and the no-large-elimination discipline is its singleton-elimination criterion. The idea of an erasable, linear token that authorizes access to a separately-held, freely-copied resource is the capability/location split of L^3 [3], and our propositions play the role of capabilities. The quantitative reading of erasure-as-the-zero-fragment is that of quantitative type theory [4], though SStruct keeps relevance and quantity as separate axes and we add impredicativity, which quantitative type theory does not have. Treating a points-to assertion as a linear proposition over a heap managed elsewhere is the standard move of separation logic and Iris [5].

2 The SStruct setting

We recall just enough of SStruct to fix notation. Nothing in this section is new.

Sorts. A *sort algebra* is a partial order $(\mathcal{S}, \sqsubseteq)$ with a least element ι and two distinguished down-closed subsets: W, the sorts that admit weakening (discard), and C, the sorts that admit contraction (duplication), with $\iota \in \mathsf{W} \cap \mathsf{C}$. The canonical instance has four sorts,

$$\mathsf{U}\ \text{(unrestricted)}, \quad \mathsf{R}\ \text{(relevant)}, \quad \mathsf{A}\ \text{(affine)}, \quad \mathsf{L}\ \text{(linear)},$$

ordered $\mathsf{U} \sqsubseteq \mathsf{R} \sqsubseteq \mathsf{L}$ and $\mathsf{U} \sqsubseteq \mathsf{A} \sqsubseteq \mathsf{L}$, with $\mathsf{W} = \{\mathsf{U}, \mathsf{A}\}$ and $\mathsf{C} = \{\mathsf{U}, \mathsf{R}\}$. A sort sits higher in the order as it admits *fewer* structural rules. The development is generic over any such algebra; we extend the algebra, not the canonical instance, in §4.

Relevance, contexts, and splitting. Each binding carries a relevance marker, i (implicit) or e (explicit), and a sort. A context Δ is a list of bindings $x :^{r}_{s} A$. Explicit bindings are the tracked resources; the ternary split relation $\Delta = \Delta_1 \boxplus \Delta_2$ shares an explicit binding only when its sort admits contraction, and otherwise sends it to one side with the other retaining it implicitly. The predicate $\mathrm{Lower}(\Delta, s)$ holds when every explicit binding of Δ has sort $\sqsubseteq s$, and bounds closure capture. An explicit binding may be *dropped* only when its sort lies in W; the sole exception is a provably dead branch, where ex-falso may drop anything. Implicit bindings are invisible to variable lookup and are duplicated freely by the split; they are the erased, untracked, specificational fragment.

The two judgments. SStruct uses a *logical* judgment with ordinary, resource-free contexts, used for type formation and for checking implicit operands, and a *program* judgment whose contexts are split and tracked. An implicit operand is typed in the logical projection of the program context, so it consumes no resources and may mention program variables freely. We write Δ° for the projection that sets every relevance marker to i (the all-implicit, or logical, view), and fold the two judgments into a single $\Delta \vdash m : A$ whose logical reading is recovered at Δ°.

Universes and the existing erasable fragment. Runtime types live in a predicative hierarchy $\mathcal{U}^{i}_{s}$ with $\mathcal{U}^{i}_{s} : \mathcal{U}^{i+1}_{\iota}$, and a dependent product lands in $\mathcal{U}^{\max(i_A, i_B)}_{s}$. Equality and falsity live in the least sort ι; their proofs are checked logically, the rewrite rule rewrites a type and erases its equality proof, and ex-falso eliminates the empty type into an arbitrary motive, extracts to a dead marker, and drops the resources of its unreachable branch. The extension folds them into the unrestricted propositional universe $\mathbb{P}_{\mathsf{U}}$ of §4 as its singleton and empty instances, with the rewrite and ex-falso rules unchanged.

3 Design: one ledger, erased by universe

3.1 Two axes, two erasures

SStruct already carries two independent annotations on every binding: a relevance marker and a sort. They drive two independent erasures.

Relevance erasure. An i binding of runtime type is an implicit argument. It is untracked and extracted to `null`, the existing implicit-calculus mechanism.

Universe erasure. A binding whose type is a proposition, one living in a universe $\mathbb{P}_s$ (§4), is a proof. Whatever its relevance marker, extraction deletes it because its type is a proposition, following Rocq's extraction mechanism.

A proof is therefore an *explicit* binding of propositional type. Being explicit, it is tracked by the ordinary split: a linear capability has the linear sort, so the split refuses to contract it, just as it refuses to duplicate a linear runtime value. Proofs and resources share one discipline, reused without modification. Having a propositional type, the proof is erased. The two facts are independent, and both ride on fields that already exist, so the design needs no third mode and no second ledger.

Remark 3.1 (Where this differs from making proofs implicit)**.** An implicit binding lives in the resource-free logical view, where it is freely duplicable, and a linear proof cannot live there without losing its linearity. The single-ledger design leaves proofs *explicit* and moves their erasure onto the separate, universe-directed pass. Tracking implicits would not work, since it would break implicit *type* arguments, which must stay freely usable at every sort.

3.2 Resource confinement

One ledger means a proof and a resource are split by the same relation, so in principle a proof could consume a runtime resource. If it did, universe-directed erasure would delete the proof and orphan the resource: a leak for a linear or affine resource, an unmet obligation for a relevant one. The design forbids this with a single principle, stated once and enforced in two places.

Principle 3.2 (Confinement)**.** *Consuming a runtime resource in the course of producing a proof is a* drop*. Consequently only a runtime resource whose sort lies in* W *may be consumed into a proof, outside a provably dead branch; a relevant or linear runtime resource may not. Proofs thus hold no non-discardable runtime resource, and erasing a proof frees nothing the runtime accounting still owes.*

Confinement reuses the drop discipline already in the calculus: a drop requires a weakenable sort and a dead branch may drop anything, and nothing about that changes. What is new is recognizing the two syntactic routes by which a resource could flow into a proof, and closing each.

Route one: a quantifier over a resource. A proposition may quantify over a runtime value. If it binds that value *explicitly*, a proof of the proposition is a function that, when applied, consumes the value. The following exhibits the leak.

Example 3.3 (Why the binder must be implicit)**.** Let *File* be linear. The product $Q \coloneqq \Pi\,(x :^{\mathsf{e}}_{\mathsf{L}} \mathit{File}).\,P$ is a proposition whenever P is, by the impredicative rule of §4. A proof $q : Q$ is unobjectionable in isolation. But with a linear $h : \mathit{File}$ in scope, the application $q\,h$ is a proof of P that *consumes* h, and universe-directed erasure then deletes $q\,h$ together with the only use of h. The handle is allocated and never freed.

The fix is a formation condition: a propositional quantifier over a *runtime* domain must bind that domain implicitly. Then Q can only be $\Pi\,(x :^{\mathsf{i}}_{\mathsf{L}} \mathit{File}).\,P$, and the application $q\,h$ passes h implicitly, as a reference rather than a consumption, so h remains on the ledger to be consumed by runtime code. The proposition references h without owning it. This is the capability/location split, where the location is referenced and the proof does not hold the resource.

Route two: an elimination that pays into a proof. An eliminator for a runtime scrutinee may have a propositional motive: one proves a proposition by case analysis on a value. If the scrutinee is linear, the elimination consumes it to produce a proof, which is then erased, the same leak by a different door. Confinement classifies this elimination as a drop of the scrutinee, so it is admissible only when the scrutinee's sort lies in W or the branch is dead. Eliminating an unrestricted

boolean into a propositional motive is fine, because booleans are weakenable. Eliminating a linear pair into a propositional motive is not, because that would discard the pair for no runtime gain. Both routes reduce to the same notion of a drop.

Remark 3.4 (Structures that embed proofs need no new rule)**.** A runtime structure may contain a proof component. A refinement $\{x : V \mid P\}$ is a runtime pair whose second component is a proof. No special condition is needed. The existing requirement that a pair's sort dominate its components' sorts already forces a structure containing a linear proof to be itself linear, hence non-discardable, so the proof inside cannot be dropped by dropping the structure. This works because the proof's sort lives in the same lattice as runtime sorts (§4).

4 The propositional universes and their placement

To place an impredicative LProp in the lattice of sorts, separate the two coordinates that "the sort" of a SStruct universe conflates: the *lattice* coordinate, which says which discipline governs the inhabitants, and the *level* coordinate, which says where in the predicative hierarchy the universe sits. Impredicativity constrains the level coordinate. The runtime/proposition distinction lives on the universe, not on the lattice. We therefore reuse the sort lattice unchanged and add, for every discipline, a propositional universe at the bottom level.

Definition 4.1 (Propositional universes)**.** For each sort $s \in \mathcal{S}$, the same lattice that governs runtime types, there is a universe $\mathbb{P}_s$ of s-disciplined erasable propositions, classified at the bottom of the hierarchy:

$$\frac{\Delta^\circ \vdash \diamond \qquad s \in \mathcal{S}}{\Delta^\circ \vdash \mathbb{P}_s : \mathcal{U}^1_\iota} \text{ (U-Prop)}$$

A proposition is a term P with $\Delta \vdash P : \mathbb{P}_s$. Its proofs are erased by the pass of §7 and tracked under the discipline of s, so $\mathbb{P}_\mathsf{L}$ is linear, $\mathbb{P}_\mathsf{A}$ affine, $\mathbb{P}_\mathsf{R}$ relevant, and $\mathbb{P}_\mathsf{U}$ unrestricted, the analogue of Rocq's `Prop`. The classifier sits at the least sort ι rather than at s. The universe $\mathbb{P}_s$ is itself an unrestricted object, while the discipline s governs its *inhabitants*. Both $\mathbb{P}_s$ and the bottom runtime universe $\mathcal{U}^0_\iota$ are classified by $\mathcal{U}^1_\iota$, the placement of `Prop` and `Set` together at the foot of the tower.

Remark 4.2 (Where in the lattice, resolved)**.** A proposition and the runtime type of the same discipline share their sort: a linear capability and a linear handle are both at L. This is correct. Everything that reads the sort, namely the context split, Lower, and the drop rule, treats a capability the way it treats a linear resource, since for duplication, discard, and capture they obey the same discipline. The capture rule, for instance, forbids a duplicable closure from holding a linear capability ($\mathsf{L} \not\sqsubseteq \mathsf{U}$) without any added provision. A proposition is distinguished from a runtime type by the *universe* it inhabits, $\mathbb{P}_s$ versus $\mathcal{U}^i_s$, and only that is consulted by the erasure pass and the two confinement conditions of §3. There is no second lattice and no cross-stratum order. The runtime/proposition boundary lives on the universe axis, and a proof is kept from holding a runtime resource by Confinement (§3) rather than by the capture rule.

Placing every $\mathbb{P}_s$ at $\mathcal{U}^1_\iota$ keeps the runtime hierarchy undisturbed. Adding the propositional disciplines multiplies the index on $\mathbb{P}$ without adding levels to the tower, and the impredicativity lives in the product rule.

Definition 4.3 (Impredicative product and pair)**.** Propositions are closed under dependent products and pairs with domains of arbitrary sort and level:

(Prop-Π)
$$\frac{\Delta^\circ \vdash A : \kappa \qquad \Delta^\circ,\ x :^r_{s_A} A \vdash P : \mathbb{P}_s \qquad \kappa \in \{\mathcal{U}^{i_A}_{s_A},\ \mathbb{P}_{s_A}\} \qquad \kappa = \mathcal{U}^{i_A}_{s_A} \Rightarrow r = \mathsf{i}}{\Delta^\circ \vdash \Pi\,(x :^r_{s_A} A).\,P : \mathbb{P}_s}$$

(Prop-Σ)
$$\frac{\Delta^\circ \vdash A : \kappa \qquad \Delta^\circ,\ x :^r_{s_A} A \vdash P : \mathbb{P}_s \qquad \kappa \in \{\mathcal{U}^{i_A}_{s_A},\ \mathbb{P}_{s_A}\} \qquad \kappa = \mathcal{U}^{i_A}_{s_A} \Rightarrow r = \mathsf{i}}{\Delta^\circ \vdash \Sigma\,(x :^r_{s_A} A).\,P : \mathbb{P}_s}$$

The conclusion is in $\mathbb{P}_s$ regardless of i_A: this is impredicativity. The side condition $\kappa = \mathcal{U}^{i_A}_{s_A} \Rightarrow r = \mathsf{i}$ is the formation-level half of Confinement (Route one of §3): a proposition may depend on the value of a runtime variable but may never bind it as a consumed resource.

The rules subsume the usual connectives. Quantification over data, $\forall(x{:}A).P$ with A runtime, is Prop-Π with κ a runtime universe and $r = \mathsf{i}$ forced. Implication between propositions, $Q \Rightarrow P$ or its linear form $Q \multimap P$, is κ propositional with $r = \mathsf{i}$ or $r = \mathsf{e}$; the linearity of $\multimap$ amounts to $s = \mathsf{L}$ with an explicit binding. Multiplicative conjunction $Q \otimes P$ is Prop-Σ with both components propositional, explicit, at L. The propositional existential $\exists(x{:}A).P$ over runtime A is κ a runtime universe with $r = \mathsf{i}$ forced; its witness is held only specificationally, the analogue of Rocq's `ex`, and the reason the witness cannot later be extracted into a program (§6).

Remark 4.4 (Why the bottom)**.** Impredicativity at the bottom plus unrestricted large elimination is Girard's paradox, which is why Rocq forbids large elimination from `Prop` and why we forbid it from $\mathbb{P}_s$ (§6). The paradox *contracts* the impredicative hypothesis, so a strictly linear impredicative universe is consistent with fewer side conditions. We do not lean on this, since the family includes the unrestricted $\mathbb{P}_{\mathsf{U}}$, whose proofs may be contracted. The conservative discipline is therefore required in general, and linearity is additional tracking rather than a licence to relax it.

5 The calculus

The term language and the context machinery are those of SStruct, ranging now over the enlarged sort algebra. We state only what is reused and what is added.

5.1 Reused without change

Context splitting $\Delta = \Delta_1 \boxplus \Delta_2$, lowering $\mathrm{Lower}(\Delta, s)$, variable lookup, and the drop rule are as in SStruct. A proof is an explicit binding $x :^{\mathsf{e}}_s P$ with $s \in \mathcal{S}$ and $P : \mathbb{P}_s$, and these rules treat it like any explicit binding. Three consequences follow, each a corollary of an existing rule applied to a proof.

- A linear proof, at $\mathsf{L} \notin \mathsf{C}$, is not contracted by the split, so a capability cannot be duplicated.
- A linear or relevant proof, at a sort outside W, cannot be dropped, so a capability that is never spent is a type error, like an unused linear handle. An affine assertion may be forgotten, while a relevant one may not.
- A closure capturing a capability obeys Lower at the capability's sort, the same way it would for a runtime resource of that sort, so a duplicable closure cannot hold a linear capability ($\mathsf{L} \not\sqsubseteq \mathsf{U}$).

Remark 5.1 (A proof closure holds no runtime resource)**.** An LProp lambda is kept from capturing a resource by Confinement (§3). A sort barrier does not do the job, because a proof closure and a runtime closure of the same discipline share a sort, so Lower does not separate them. A free variable becomes an *explicit* capture only if the body consumes it, and a proposition-typed body consumes a runtime resource only as a drop, so it does not consume a non-weakenable one at all. A proof closure can therefore hold a runtime resource only of weakenable sort, which erasure may discard, never the linear or relevant resource that would leak. A linear or relevant runtime resource can appear in the body only implicitly, by reference; an implicit occurrence is erased and leaves the variable on the ledger, owed to the enclosing code. A proof never holds the resource it names. Captures of proofs are a separate matter: a proof closure may hold and consume other proofs, bounded by the ordinary Lower discipline, since those captures are themselves erased and own no resource. The parameter side is the dual Route-one condition on (Prop-Π). The shared sort also gives the capability-capture discipline at no cost: a capability sits at L, so a duplicable closure cannot hold one, just as it cannot hold a linear handle.

Example 5.2 (A capture leak, rejected)**.** Let *File* be linear, and let a handle $h :^{\mathsf{e}}_{\mathsf{L}} \mathit{File}$ be in scope. Attempt to hide h inside a proof by abstracting an arbitrary proposition Q and consuming h in the body:

$$p \;\coloneqq\; \lambda\_.\, m \;:\; \Pi\,(\_ :^{\mathsf{e}}_{s_Q} Q).\, P, \qquad Q : \mathbb{P}_{s_Q}, \quad P : \mathbb{P}_s, \quad s \in \mathcal{S}.$$

For p to capture h, the body $m : \mathbb{P}_s$ must consume it. Every way a proposition-typed term consumes a value runs through a route Confinement closes. An application or pairing whose result is a proposition forces a runtime argument implicit (Prop-Π, Prop-Σ), and an elimination into a propositional motive is admissible only as a drop, which $\mathsf{L} \notin \mathsf{W}$ forbids (§6). So m cannot consume h; it can mention h only implicitly, $h :^{\dot{}}_{\mathsf{L}} \mathit{File}$, which does not consume it. The handle stays on the ledger, to be spent by runtime code, and erasing p frees nothing. Here $\mathrm{Lower}(\Delta, s)$ does *not* reject the derivation; with $s = \mathsf{L}$ it would even permit h as a capture. The block is that no well-typed body consumes h in the first place. The *read* capability of §9 is the contrast: the proof only *names* the cell, through an implicit index, so it never tries to hold it.

5.2 Formation, introduction, elimination

Type formation adds (U-Prop), (Prop-Π), and (Prop-Σ) of §4. Introduction reuses the abstraction and pairing rules; because the propositional products were formed in the all-implicit view and carry the Route-one side condition, no separate introduction-time check is needed. The abstraction rule, for reference, is the SStruct rule unchanged:

$$\text{(Π-I)}\quad \frac{\mathrm{Lower}(\Delta, s) \qquad \Delta^{\circ} \vdash A : \kappa \qquad x :^{r}_{s_A} A, \Delta \vdash m : B}{\Delta \vdash \lambda x.\, m : \Pi\,(x :^{r}_{s_A} A).\, B}$$

which types a proof of an implication when $B : \mathbb{P}_s$ and a function when $B : \mathcal{U}^i_s$, with the Lower premise bounding capture on the one ledger. Application reuses the existing explicit and implicit rules. The new use is application of a runtime function to a *proof* argument,

$$\text{(Π-E)}\quad \frac{\Delta = \Delta_1 \boxplus \Delta_2 \qquad \Delta_1 \vdash m : \Pi\,(x :^{\mathsf{e}}_{s_A} A).\, B \qquad \Delta_2 \vdash n : A}{\Delta \vdash m\, n : B[n/x]}$$

read with $A : \mathbb{P}_{s_A}$: the split charges Δ_2 for the proof n, linearly when $s_A = \mathsf{L}$, which is how a runtime computation spends a capability. The elimination forms carry the Route-two condition of §6.

6 Elimination: the two walls

Eliminating across the runtime/proposition boundary is restricted in both directions, for two different reasons.

6.1 The progress wall: proposition into runtime

Progress after erasure is the requirement that a closed, well-typed runtime term never gets stuck. A proof erases to nothing; if an elimination fed that absence into a position the runtime inspects, evaluation would jam. We restrict the *motive* of every propositional eliminator, recovering the singleton-elimination criterion of Rocq and Lean.

Definition 6.1 (Extractable content)**.** A proposition $P : \mathbb{P}_s$ is *empty* if it has no constructors. It is *singleton* if it has exactly one constructor whose fields are all themselves erased, that is, of propositional type or implicit, so that the constructor stores no explicit runtime data. Otherwise it is *general*.

Definition 6.2 (Progress wall)**.** The eliminator of $P : \mathbb{P}_s$ with motive C is admissible as follows. If P is empty or singleton, C may be any type, runtime or propositional. If P is general, C must be propositional.

An empty proposition has no closed proof, so its eliminator into a runtime motive is never reduced; this is the existing ex-falso, which extracts to a dead marker. A singleton has a unique proof up to its erased fields, so eliminating it reveals no runtime information and acts as a coercion, the existing rewrite. A general proposition carries information in which-constructor, and reading it into a runtime result would require the erased proof at runtime, so its motive is confined to the propositional world and the eliminator erases wholesale.

Remark 6.3 (The existential's trapped witness)**.** $\exists(x{:}A).P$ over runtime A stores the witness as explicit runtime data, so it is not singleton, and Definition 6.2 forbids eliminating it into a runtime motive: one cannot project the witness into a program. The witness was held implicitly and is gone after extraction, so no runtime computation may depend on it. This is Rocq's restriction on `ex`.

Remark 6.4 (Linearity at a singleton elimination)**.** Large elimination of a singleton still threads the ledger. If the singleton's erased fields are linear proofs, the eliminator accounts for them through the split, so a linear hypothesis released by the elimination is spent once. Well-founded recursion is the main case: an accessibility predicate is a singleton, its recursive subproofs are propositional, and recursion eliminates it into a runtime motive while the ledger forces each subproof to be used in a structurally decreasing way. The accessibility proof is erased, and the recursion still runs.

6.2 The confinement wall: runtime into proposition

The opposite direction is sound for progress but unsound for resources, and is governed by Confinement (Route two, §3).

Definition 6.5 (Confinement wall)**.** An eliminator for a runtime scrutinee m with a propositional motive $C : \mathbb{P}_s$ is admissible only when m's sort lies in W, or the elimination sits in a provably dead branch. The elimination is then accounted as a drop of m.

The two walls are independent. The progress wall constrains the motive when the *scrutinee* is a proposition; the confinement wall constrains the scrutinee when the *motive* is a proposition. Together they allow the boundary between the runtime world and the propositional world to be crossed only in ways that strand neither a computation nor a resource.

7 Erasure and the runtime

Extraction $\lfloor \cdot \rfloor$ maps a well-typed program term to a runtime term by two deletions: the relevance deletion of SStruct, which sends implicit runtime arguments to `null`, and the propositional deletion, which removes every subterm whose type is propositional. The propositional deletion generalizes the per-construct erasure already present, where the rewrite extracts to its body and ex-falso to a dead marker, into a uniform rule. Because SStruct's extraction is defined over the typing derivation, the type of each subterm is available and the pass is well-defined.

- A proof in argument position erases to `null`; a proof in a runtime structure erases to a hole, the structure surviving.
- A proof abstraction, application, pair, projection, and any general eliminator erase wholesale, as the rewrite already does.
- An empty elimination erases to a dead marker, its dead branch's live runtime resources wrapped in `drop`; its propositional resources contribute no `drop`, having no runtime presence.

The runtime semantics is untouched, because propositional subterms are gone before the runtime layer runs. That semantics is a heap of sorted cells, with sharing permitted only for contractible sorts and linear cells droppable only in dead branches.

Lemma 7.1 (No propositional allocation)**.** *If $\Delta \vdash m : P$ with $P : \mathbb{P}_s$, then $\lfloor m \rfloor$ contains no allocating form and reduces to no heap cell.*

Proposition 7.2 (Erasure preserves drop-safety)**.** *Extending* SStruct *with* LProp *leaves the heap invariants intact. By Lemma 7.1 a proof occupies no cell, so erasing it frees nothing; the runtime ledger and its drop-safety argument are those of the unextended system.*

This is the sense in which LProp cannot leak. The only resources on the heap are runtime resources, and by Confinement no proof consumed a non-discardable one, so there is nothing to leak.

8 Metatheory

We list the obligations a formalization would discharge. Two are central: the resource-confinement lemma, which the single-ledger design must prove where the two-ledger alternative of §11 would get it structurally, and consistency of the impredicative universe.

Lemma 8.1 (Resource confinement)**.** *In a well-typed term, every runtime resource consumed in producing a proposition-typed subterm has a sort in* W*, unless the consumption lies in a provably dead branch. Equivalently, deleting all proposition-typed subterms leaves the runtime resource accounting unchanged.*

Strategy. Induction on typing. A runtime variable can enter a proposition-typed subterm only through a quantifier domain or an elimination scrutinee. The formation condition of Definition 4.3 forces the former to be implicit, hence consumed as no resource, and the confinement wall of Definition 6.5 forces the latter to be a drop, hence weakenable or dead. No other rule moves a runtime explicit binding into a propositional conclusion. This lemma is what makes Proposition 7.2 hold, and it is the cost of using a single ledger.

Theorem 8.2 (Preservation)**.** *If $\Delta \vdash m : A$ and $m \longrightarrow m'$ then $\Delta \vdash m' : A$.*

Strategy. The SStruct argument, with substitution and inversion extended to the propositional universes. Substituting a proof for an explicit propositional variable uses the same split as substituting a value for an explicit runtime variable. The formation and confinement conditions are preserved because they are conditions on sorts, which reduction does not change.

Theorem 8.3 (Progress after erasure)**.** *If $\vdash m : A$ in the empty context then $\lfloor m \rfloor$ is a value or steps.*

Strategy. The only new stuck configuration would be a runtime elimination scrutinizing an erased proof; Definition 6.2 excludes it, its empty case by the closed uninhabitedness argument already in SStruct, its singleton case by erasure to a coercion that does not scrutinize.

Theorem 8.4 (Erasure simulation)**.** *If $\Delta \vdash m : A$ and $m \longrightarrow m'$, then $\lfloor m \rfloor = \lfloor m' \rfloor$ when the step is internal to a proposition or to an implicit, and $\lfloor m \rfloor \longrightarrow \lfloor m' \rfloor$ otherwise.*

Strategy. By cases on the step, using Lemma 7.1 and the fact that propositional subterms occur only in positions the propositional deletion discards.

Conjecture 8.5 (Consistency)**.** *The logical layer with the impredicative family $\mathbb{P}_s$ and the progress wall of Definition 6.2 is consistent: there is no closed proof of $\mathbf{0}$.*

Strategy, and the hardest point. Impredicativity is not a structural induction on levels, so this needs a model of the impredicative universe in the style of reducibility candidates, rather than the predicative normalization SStruct uses. Two observations make it tractable. The substructural fragment is a restriction of the unrestricted one: forgetting the tracking turns any linear, affine, or relevant proof into a valid $\mathbb{P}_{\mathsf{U}}$ proof, so it suffices to establish consistency of the unrestricted impredicative universe, ordinary impredicative `Prop`, and inherit it by the forgetful inclusion, as quantitative type theory relates to its underlying theory. The no-large-elimination discipline makes the model possible, since a proposition may be interpreted without committing to runtime content. This is the one part that requires real work; the rest is adaptation.

9 Examples

A linear capability for a mutable cell. Let $\mathit{Ref} : \mathcal{U}^0_{\mathsf{U}}$ be copyable cell names and let $\mathit{Pts} : \mathit{Ref} \to V \to \mathbb{P}_{\mathsf{L}}$ be the points-to assertion, a linear proposition. Reading requires the capability and returns it:

$$\mathit{read} : \Pi\,(r :^{\mathsf{i}}_{\mathsf{U}} \mathit{Ref}).\,\Pi\,(v :^{\mathsf{i}}_{\mathsf{U}} V).\,\Pi\,(c :^{\mathsf{e}}_{\mathsf{L}} \mathit{Pts}(r, v)).\ (\Sigma\,(x :^{\mathsf{e}}_{\mathsf{U}} V).\,\mathit{Pts}(r, v)).$$

The indices r, v are implicit by the Route-one condition, since *Pts* only *names* the cell, and they erase. The capability c is explicit and linear, so the split spends it and the type system forbids two writers, yet c has propositional type and erases to `null`. At runtime *read* is a bare load. The cell itself is a runtime resource, owned by the runtime ledger and unaffected.

Refinement. $\{x : V \mid P\}$ is the runtime pair $\Sigma\,(x :^{\mathsf{e}}_{\mathsf{U}} V).\,P$ with $P : \mathbb{P}_s$ a proof component. It erases to V. If P is linear the component-sort condition forces the pair's sort up to a linear runtime sort, so the pair cannot be silently dropped, and the proof inside is spent honestly.

Equality and falsity, recovered. Equality is the singleton proposition Eq : $\mathbb{P}_{\mathsf{U}}$, whose sole constructor stores no explicit data, so Definition 6.2 permits elimination into a runtime motive and extraction deletes the proof. This is the existing rewrite, now an impredicative equality pinned at $\mathbb{P}_{\mathsf{U}}$ rather than floating to the level of its type. Falsity is the empty proposition $\mathbf{0} : \mathbb{P}_{\mathsf{U}}$, eliminable into any motive and extracting to a dead marker, the existing ex-falso. The rewrite and ex-falso rules are the singleton and empty instances and need no change.

10 Application: amortized cost analysis

The linear propositions give capabilities and separation logic. The *affine* propositions give, through the same erased-and-tracked discipline, a static cost analysis in the style of the potential method, namely automatic amortized resource analysis [6, 7], with the potential carried by erased affine propositions and the bound certified by typing. The carrier $P\,n$ is the time-credit assertion of separation logic [8] reread as an erased affine type.

10.1 Potential and its algebra

Let a *potential* be an affine proposition indexed by an amount, $P : \mathbb{N} \to \mathbb{P}_{\mathsf{A}}$, where a value of $P\,n$ is ownership of n units of an abstract, erased credit. Affineness is the right discipline. Since $\mathsf{A} \notin \mathsf{C}$, a token cannot be duplicated, so credits cannot be forged and the bound is sound. Since $\mathsf{A} \in \mathsf{W}$, a token may be dropped, so the obligation is "cost $\leq$ budget" rather than "cost = budget". Linear potential would force every credit to be spent; unrestricted potential would let credits be duplicated, and the bound would collapse. Affine is the only sort that gives the amortized inequality.

The operations on potential form a graded commutative monoid, packaged as a class over the carrier (with $\multimap$ the affine function, an explicit binder at A):

$$\begin{aligned}&\textbf{class}\ \mathrm{Potential}\,(P : \mathbb{N} \to \mathbb{P}_{\mathsf{A}})\ \textbf{where}\\ &\quad \mathit{nil} : P\,0\\ &\quad \mathit{split} : \forall\, a\, b.\ P\,(a{+}b) \multimap P\,a \otimes P\,b\\ &\quad \mathit{join} : \forall\, a\, b.\ P\,a \otimes P\,b \multimap P\,(a{+}b)\end{aligned}$$

Spending is not primitive. The unit-cost step $\mathit{use} : P\,(n{+}1) \multimap P\,n$ is derived by splitting off a single credit and discarding it,

$$\mathit{use}\,p \;=\; \textbf{let}\ (\_, r) = \mathit{split}\,p\ \textbf{in}\ r,$$

the discard being the affine weakening that $\mathsf{A} \in \mathsf{W}$ permits. SStruct has no surface **drop**: a weakenable value is discarded by leaving it unused, and extraction synthesizes whatever runtime `drop` node is needed, here none, since the credit is erased. Spending a credit is therefore *affine discard*, *split* is the sole primitive, and a k-cost step $P\,(n{+}k) \multimap P\,n$ is the same construction at width k. The class is surface convenience: [Potential P] desugars to an implicit dictionary argument, used specificationally. The operations are reusable rules, and only the $P\,n$ tokens are the affine currency.

A *cost point* is any operation whose type consumes a credit; the implementor fixes a cost model by choosing which operations carry one. Tying the heap interface of §9 to potential, as in

$$\mathit{write} : \forall\, n\, r\, v\, v'.\ P\,(n{+}1) \multimap \mathit{Pts}(r, v) \multimap V \multimap \mathit{Pts}(r, v') \otimes P\,n,$$

makes the same erased potential bound the number of heap writes, recovering the original heap-space target of the method [6]. Capabilities certify that the writes are safe, and potential bounds how many there are.

10.2 The soundness criterion

A method may be added to the class when it is *potential-non-increasing*: the total amount in its result is at most the total in its argument. For a method of type $\bigotimes_i P\, a_i \multimap \bigotimes_j P\, b_j$ the condition is $\sum_j b_j \le \sum_i a_i$, with *nil* admitted because it introduces the amount 0. Every operation above conserves the total (*split*, *join*) or introduces nothing positive (*nil*), so no derivation can raise the total potential above the seed. The condition is visible in the $\mathbb{N}$ indices of the method types. The monoid laws, associativity and commutativity of *split*, are for reasoning and play no part in the bound.

10.3 Analyzing a function

A computation is made cost-analyzable by abstracting the carrier and demanding the class:

$$f : \forall\,(P : \mathbb{N} \to \mathbb{P}_{\mathsf{A}})\,[\mathrm{Potential}\,P].\ P\,(\mathit{cost}\ \mathit{xs}) \multimap \mathit{Args} \to B,$$

where *cost* is the claimed bound as a function of the inputs. Two properties make this work.

The bound is a fact about typing at the abstract P. Because P is a rigid parameter, the body has no introduction form for $P\,k$ beyond the seed and the class methods, the same abstraction that traps an existential witness in §6, and by the non-increasing criterion the indices never climb above $\mathit{cost}\ \mathit{xs}$. Once f type-checks, the number of cost points along any run is fixed at $\le \mathit{cost}\ \mathit{xs}$, and the caller may instantiate P at the top with *any* dictionary, even a trivial phantom, without changing it. The guarantee is the free theorem for the quantified P. It is metatheoretic, established once in the same reducibility-candidate model that Conjecture 8.5 requires, instrumented to count credits, rather than proved per program. The implementor's "proof that the potential covers the cost" is simply that the body type-checks. The failure mode is to specialize P to a concrete, forgeable type *before* checking, so keeping P abstract is what keeps the guarantee.

The accounting is ghost. The carrier P is a type family and $\mathit{nil}, \mathit{split}, \mathit{join}$ are affine proofs, so the dictionary and every $P\,n$ token are erased by the pass of §7, and f extracts to $\mathit{Args} \to B$ with no trace of the analysis. Potential composes because the seed is indexed by the inputs and the constraint propagates along the call graph: a caller and the callees it invokes share the one currency P, so a budget established at the top is parcelled out through *split* and spent at the leaves, and the type of f states the resulting amortized bound.

11 Design alternatives

The two-ledger alternative. One may instead give proofs a separate ghost ledger and a third relevance mode, erased and tracked, distinct from both implicit and explicit. Confinement then holds structurally, since no rule moves a runtime resource into the ghost ledger, so Lemma 8.1 is by construction rather than by induction. The cost is a third mode threaded through every context operation and a second split relation. The single-ledger design trades that machinery for one inductive lemma, and reuses extraction by universe, whose metatheory is well understood. For a codebase that already has the split, the lower-machinery option is preferable. The two-ledger design is the better choice only if the propositional layer is expected to diverge sharply from the runtime discipline.

Predicative propositions. If impredicativity is not wanted, propositions may instead float up with the level of what they quantify over, as equality already does. This discharges Conjecture 8.5 by predicative normalization, at the cost of quantification over all propositions. The propositional

universes, the confinement principle, and the two walls are unchanged; only (U-Prop) and the product rule change.

One sort or the family. Reusing the sort lattice to index the propositional universes costs almost nothing over providing only $\mathbb{P}_{\mathsf{L}}$, and it adds affine and relevant propositions: affine for assertions that may be forgotten but not duplicated, relevant for obligations that may be duplicated but must be met. We recommend the family.

Definitional proof irrelevance. Orthogonally, $\mathbb{P}_s$ could be made definitionally proof-irrelevant, as Lean's `SProp` is. This strengthens conversion but is in tension with linear tracking, since irrelevance would equate a used and an unused proof; we record it as a deliberate non-goal.

12 Summary

The extension adds a propositional universe $\mathbb{P}_s$ for every sort of the existing lattice, placed at the bottom of the level hierarchy and quantified impredicatively, so that linear, affine, relevant, and unrestricted propositions arise uniformly. A proof is an ordinary explicit variable of propositional type. The existing context split tracks it under the discipline its sort names, and a separate universe-directed pass erases it, as Rocq's extraction erases `Prop`, so one ledger suffices. That kind of erasure is sound in Rocq because every value is discardable. The only substructural addition is the confinement principle, that a runtime resource is consumed into a proof only as a drop, enforced at quantifier formation and at elimination into a propositional motive, and reusing the existing drop discipline with its dead-branch exception. Progress after erasure is preserved by the singleton-elimination criterion transported to the substructural setting. The existing equality and falsity become the singleton and empty inhabitants of $\mathbb{P}_{\mathsf{U}}$, with their rules unchanged. The two substantial proof obligations are the resource-confinement lemma, which the single ledger pays for by induction, and consistency of the impredicative universe, which reduces to that of ordinary impredicative `Prop` by forgetting the substructural tracking.

References

[1] B. Barras and B. Bernardo. *The Implicit Calculus of Constructions as a Programming Language with Dependent Types.* FoSSaCS, 2008.

[2] P. Letouzey. *A New Extraction for Coq.* TYPES, 2002.

[3] A. Ahmed, M. Fluet, and G. Morrisett. *L^3: A Linear Language with Locations.* Fundamenta Informaticae, 2007.

[4] R. Atkey. *Syntax and Semantics of Quantitative Type Theory.* LICS, 2018.

[5] R. Jung et al. *Iris from the ground up: A modular foundation for higher-order concurrent separation logic.* JFP, 2018.

[6] M. Hofmann and S. Jost. *Static prediction of heap space usage for first-order functional programs.* POPL, 2003.

[7] J. Hoffmann, K. Aehlig, and M. Hofmann. *Multivariate amortized resource analysis.* TOPLAS, 2012.

[8] A. Charguéraud and F. Pottier. *Verifying the correctness and amortized complexity of a union-find implementation in separation logic with time credits.* JAR, 2019.